

Gourme Final Report



Submitted By

Mehraeel Philip

Fatma Milad

Sama Gehad

Service Oriented Final Project

May 2026

1 Problem Definition

1.1 Business Challenge

Restaurants have a difficult time handling online order management, logistics of delivery operations and keeping customers engaged. Traditional phone systems used for ordering are slow, have high error rates and do not provide real-time tracking of food deliveries.

1.2 Solution Overview

Gourme is an all-inclusive restaurant ordering system that is based on a microservices approach and allows:

- Customers to search for restaurant menus, place orders, track their orders as they are being delivered in real-time.
- Restaurant administrators can administer the restaurant's menu items, view order progress, and schedule delivery service.
- Seamless integration of the ordering, payment and delivery systems.

1.3 Key Problems Solved

Problem	Gourme Solution
Manual order taking	Digital menu with instant ordering
No delivery tracking	Real-time GPS-based delivery tracking
Disconnected systems	Integrated microservices architecture
No customer feedback	Built-in rating and review system
Inventory management	Admin dashboard for menu management

2 Project Scope

Customer Features

- User registration and authentication
- Browse menu with category filtering
- Add items to shopping cart

- Checkout with delivery address and payment method
- View order history
- Real-time delivery tracking
- Submit feedback with star ratings

Admin Features

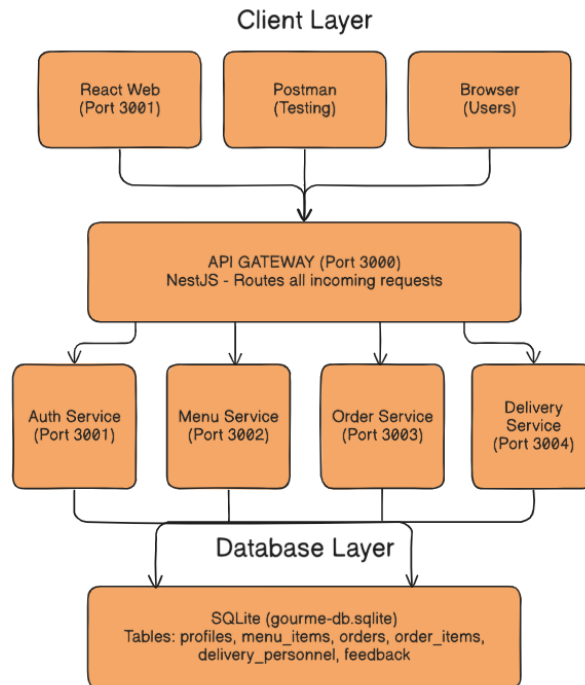
- Dashboard with analytics (sales, orders, ratings)
- Menu item management (CRUD operations)
- Order management and status updates
- Delivery personnel management
- Customer feedback review

Technical Features

- Microservices architecture with API Gateway
- JWT-based authentication
- RESTful API design
- SQLite database with TypeORM
- React frontend with Tailwind CSS

3 Architecture Overview

3.1 System Architecture Diagram



3.2 Microservices Communication Flow

Order Creation Flow:

1. User adds items to cart (Frontend)
2. User checks out (POST /api/orders)
3. API Gateway routes to Order Service
4. Order Service creates order in database
5. Admin updates order status
6. Delivery Service assigns driver
7. Customer tracks delivery (GET /api/delivery/status)

3.3 Technology Stack

3.3.1 Backend Technology

Technology	Version	Purpose
NestJS	10.x	Backend framework for microservices
TypeORM	0.3.x	ORM for database interaction
SQLite3	5.1.x	Embedded database (development)
JWT	10.x	Authentication tokens
bcrypt	5.1.x	Password hashing
Axios	1.6.x	HTTP client for service communication

3.3.2 Frontend Technology

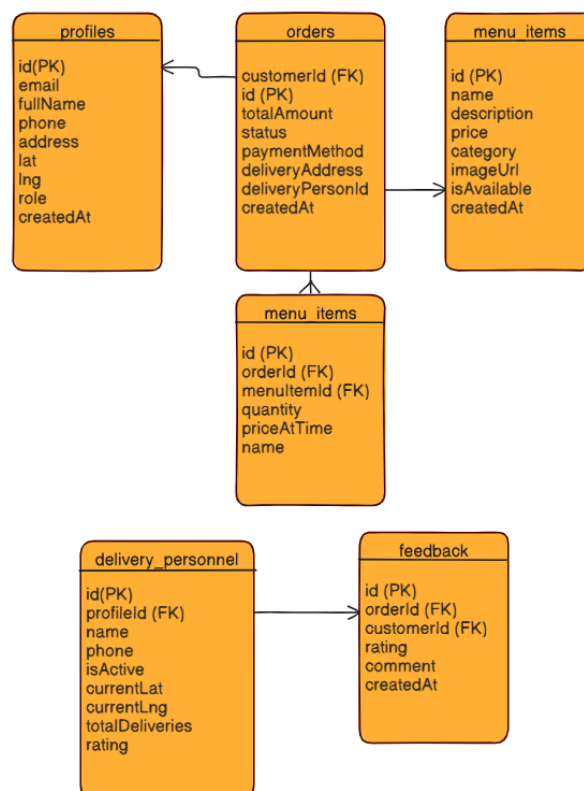
Technology	Version	Purpose
React	18.x	UI framework
TypeScript	5.x	Type-safe JavaScript
Tailwind CSS	3.4.x	Utility-first CSS framework
Axios	1.6.x	HTTP client for API calls
React Router DOM	6.x	Client-side routing
Heroicons	2.x	Icon library

3.3.3 Why SQLite?

Reason	Explanation
No internet required	Works offline, no connection issues
Zero configuration	No API keys, no setup required
Development speed	Immediate database access
Portability	Single file database, easy to share
No costs	Free, no cloud dependencies
Testing convenience	Easy to reset, no data limits

4 Database Overview

4.1 Entity Relationship Diagram (ERD)



Endpoint	Method	Description
/auth/register	POST	Create new user account
/auth/login	POST	Authenticate and receive JWT

5 Microservices Documentation

1. Auth Service (/services/auth-service)

Purpose: Handle user registration, authentication, and authorization

Request/Response Examples:

json// POST /api/auth/register

Request:

```
{ "email": "user@example.com",
  "password": "secure123",
  "fullName": "John Doe",
  "phone": "+1234567890",
  "address": "123 Main St", "role": "customer" }
```

Response:

```
{ "access_token": "eyJhbGciOiJIUzI1NiIs...\"",
  "user": {
    "id": "user_1234567890_abc",
    "email": "user@example.com",
    "fullName": "John Doe",
    "role": "customer" } }
```

2. Menu Service (/services/menu-service)

Purpose: Manage restaurant menu items

3. Order Service (/services/order-service)

Purpose: Process orders and manage order lifecycle

4. Delivery Service (/services/delivery-service)

Purpose: Manage delivery personnel and track deliveries

5. Feedback Service (/services/feedback-service)

Purpose: Collect and manage customer feedback

6 API Documentation

Authentication Endpoints

#	Method	URL	Description	Auth Required
1	POST	/api/auth/register	Create account	No
2	POST	/api/auth/login	Login user	No

Menu Endpoints

#	Method	URL	Description	Auth Required
3	GET	/api/menu	Get all items	No
4	GET	/api/menu/available	Get available items	No
5	GET	/api/menu/category/:category	Get by category	No
6	GET	/api/menu/:id	Get single item	No
7	POST	/api/menu	Create item	Admin
8	PUT	/api/menu/:id	Update item	Admin
9	DELETE	/api/menu/:id	Delete item	Admin

Order Endpoints

#	Method	URL	Description	Auth Required
10	GET	/api/orders	Get all orders	Admin
11	GET	/api/orders/my-orders	Get my orders	Customer
12	GET	/api/orders/:id	Get order details	Yes
13	POST	/api/orders	Create order	Customer
14	PUT	/api/orders/:id/status	Update status	Admin
15	GET	/api/orders/stats/sales	Total sales	Admin
16	GET	/api/orders/stats/count	Order count	Admin

Delivery Endpoints

#	Method	URL	Description	Auth Required
17	GET	/api/delivery/personnel	Get all drivers	Admin
18	POST	/api/delivery/personnel	Add driver	Admin
19	PUT	/api/delivery/personnel/:id/location	Update location	Admin
20	POST	/api/delivery/assign	Assign to order	Admin
21	GET	/api/delivery/status/:orderId	Track delivery	Customer
22	PUT	/api/delivery/complete/:orderId	Complete delivery	Admin

Feedback Endpoints

#	Method	URL	Description	Auth Required
23	GET	/api/feedback	All feedback	Admin
24	GET	/api/feedback/my-feedback	My feedback	Customer
25	POST	/api/feedback	Submit feedback	Customer
26	GET	/api/feedback/stats/average	Average rating	No

7 Frontend Documentation

```
src/pages/
├── admin/
│   ├── AdminDashboard.tsx # Main dashboard with stats
│   ├── AdminMenu.tsx      # Menu management
│   ├── AdminOrders.tsx    # Order management
│   ├── AdminDelivery.tsx  # Delivery personnel
│   ├── AdminFeedback.tsx  # Customer reviews
│   ├── LoginPage.tsx      # User login
│   ├── RegisterPage.tsx   # New user registration
│   ├── MenuPage.tsx       # Browse menu with categories
│   ├── CartPage.tsx       # Shopping cart and checkout
│   └── MyOrdersPage.tsx   # Order history and tracking
```

Figure 1: Pages Frontend Structure

```
App
├── AuthProvider
├── CartProvider
├── Router
│   ├── Navbar
│   ├── Routes
│   │   ├── LoginPage
│   │   ├── RegisterPage
│   │   ├── MenuPage
│   │   ├── CartPage
│   │   ├── MyOrdersPage
│   │   └── AdminDashboard
│   │       ├── AdminMenu
│   │       ├── AdminOrders
│   │       ├── AdminDelivery
│   │       └── AdminFeedback
```

Figure 2: Component Hierarchy

8 Testing Documentation

Test Case	Status	Expected vs Actual
User Registration	PASS	Token received, user created
User Login	PASS	JWT token valid
Get Menu	PASS	14 items returned
Create Order	PASS	Order saved with items
Update Order Status	PASS	Status changes correctly
Assign Delivery	PASS	Driver assigned
Track Delivery	PASS	Real-time status shown
Submit Feedback	PASS	Feedback saved
Admin Dashboard	PASS	All stats displayed

9 Installation and Setup

9.1 Prerequisites

- Node.js (v18 or higher)
- npm (v9 or higher)
- VS Code (recommended)

9.2 Github Link

https://github.com/MiraPhilip12/Service_Oriented_Final.git

10 Tools and Technologies

10.1 Development Environment

Tool	Purpose
VS Code	Code editor with TypeScript support
Postman	API testing and documentation
Git	Version control
npm	Package management

10.2 Backend Frameworks & Libraries

Library	Purpose
NestJS	Microservices framework
TypeORM	Database ORM
SQLite	Embedded database
JWT	Authentication
bcrypt	Password hashing

10.3 Frontend Frameworks & Libraries

Library	Purpose
React	UI framework
TypeScript	Type safety
Tailwind CSS	Styling
Axios	HTTP requests
React Router	Navigation
Heroicons	Icons

11 Local Deployment Guide

1. Start backend: `cd gourme-backend && npm run start:dev`
2. Start frontend: `cd gourme-frontend && npm start`
3. Access application: `http://localhost:3001`